PROGRAMMING WITH PYTHON

Lekha A
Computer Applications

PES UNIVERSITY
CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Python Basics and Standard Library
Anonymous Functions

Lekha A
Computer Applications

PROGRAMMING WITH PYTHON

Recap

- Recursion
- Examples

PROGRAMMING WITH PYTHON

Anonymous Functions

PES UNIVERSITY
CELEBRATING 50 YEARS

- A function that is defined without a name.
- While normal functions are defined using the def keyword, anonymous functions are defined using the lambda keyword.

PROGRAMMING WITH PYTHON

Anonymous Functions

PES UNIVERSITY
CELEBRATING 50 YEARS

- The foundation of this is the work of Alonzo Church – foundations of computer science in 1936.
- His lambda calculus forms the basis for many modern functional languages.
- Hence anonymous functions are also called lambda functions.

Fig: https://en.wikipedia.org/wiki/Alonzo_Church

PROGRAMMING WITH PYTHON

Anonymous Functions

PES UNIVERSITY
CELEBRATING 50 YEARS

- Syntax
  - lambda arguments: expression

```
>>> x = lambda a : a + 10
>>> x
<function <lambda> at 0x0394FAE0>
>>> x(5)
15
```

PROGRAMMING WITH PYTHON

Use of Lambda functions

PES UNIVERSITY
CELEBRATING 50 YEARS

- Lambda functions is generally used as an argument to a higher-order function (a function that takes in other functions as arguments).
- Lambda functions are used along with built-in functions like filter(), map() etc.

PROGRAMMING WITH PYTHON

Map()

- The programmer needs to walk through an iterable, do some operation, collect the result in an iterable.
- The number of elements in the output will be same as the number of elements in the input.
  - There is a builtin function called map to do this.

PROGRAMMING WITH PYTHON

Map()

- The function map takes two arguments - a callable and an iterable.
- The map function causes iteration through the iterable, applies the callable on each element of the iterable and creates a map object which is also an iterable.

PROGRAMMING WITH PYTHON

Map()

PES UNIVERSITY
CELEBRATING 50 YEARS

- The first argument for the map function should be a callable which takes one argument – could be a free function, could be a function of a type, could be user defined function, could be a lambda function as well.

```
>>> a = [ 'apple', 'pineapple', 'fg', 'mangoes' ]
>>> b = list(map(len, a))
>>> b
[5, 9, 2, 7]
```

PROGRAMMING WITH PYTHON

filter()

- There are cases where there is a need to remove a few elements of the input iterable.
- Use the function filter.

PROGRAMMING WITH PYTHON

filter()

- The filter function has the following characteristics.
  - - input : an iterable of some # of elements (say n)
  - - output: a lazy iterable of 0 to n elements(between 0 and n)
  - - elements in the output: apply the callback on each element of the iterable – if the function returns True, select the input element and otherwise do not select the input element.

PROGRAMMING WITH PYTHON

filter()

```
>>> seq = [0, 1, 2, 3, 5, 8, 13]
>>> list(filter(lambda x: x % 2 != 0, seq) )
[1, 3, 5, 13]
>>> list(filter(lambda x: x % 2 == 0, seq) )
[0, 2, 8]
```

PROGRAMMING WITH PYTHON

Filter()

- There are many cases where there may be a need to do both mapping and filtering.
- It is always a good habit filter first and then map.
- Otherwise map will have to do processing of some elements which eventually be filtered out.

PROGRAMMING WITH PYTHON

reduce()

- The characteristics are as follows.
  - - input : an iterable of some # of elements (say n)
  - - output : a single element
  - - processing : requires a callback which takes two elements.

PROGRAMMING WITH PYTHON

reduce()

```
>>> from functools import reduce
>>> nums = [1, 2, 3, 4]
>>> ans = reduce(lambda x, y: x + y, nums)
>>> print(ans)
10
```

PROGRAMMING WITH PYTHON

zip()

- zip does not stand for data compression.
- The function zip is used to associate the corresponding elements of two or more iterables into a single lazy iterable of tuples.
- It does not have any callback function.

PROGRAMMING WITH PYTHON

zip()

PES UNIVERSITY
CELEBRATING 50 YEARS

```
>>> a=list(input("Enter the elements").split())
Enter the elements2 4 6 8 2
>>> b=list(input("Enter the elements").split())
Enter the elements1 3 5 7 9
>>> zip(a,b)
<zip object at 0x03C46990>
>>> list(zip(a,b))
[('2', '1'), ('4', '3'), ('6', '5'), ('8', '7'), ('2', '9')]
```

PROGRAMMING WITH PYTHON

max()

- This function is used to compute the maximum of the values passed in its argument and lexicographically largest value if strings are passed as arguments.
  - max(a,b,c,..)
    - a,b,c,.. : similar type of data.
  - Returns the maximum of all the arguments.
  - Returns TypeError when conflicting types are compared.

PROGRAMMING WITH PYTHON

max()

PES UNIVERSITY
CELEBRATING 50 YEARS

-

```
>>> max(12,23,45,67)
67
>>> max([12,23,45,67],[12,23,55,67],[12,23,45,77],[92,23,45,67])
[92, 23, 45, 67]

>>> max(12, 12.34)
12.34
>>> max("Hello",'hello', 'HELLO')
'hello'
```

PROGRAMMING WITH PYTHON

min()

- This function is used to compute the minimum of the values passed in its argument and lexicographically smallest value if strings are passed as arguments.
  - min(a,b,c,..)
    - a,b,c,.. : similar type of data.
  - Returns the minimum of all the arguments.
  - Returns TypeError when conflicting types are compared.

PROGRAMMING WITH PYTHON

min()

```
>>> min(12, 12.34)
12
>>> min({12,23,45,67},[12,23,55,67],{12,23,45,77},{92,23,45,67})
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    min({12,23,45,67},[12,23,55,67],{12,23,45,77},{92,23,45,67})
TypeError: '<' not supported between instances of 'list' and 'set'
>>> min(12, 12.34, 7+7j)
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    min(12, 12.34, 7+7j)
TypeError: '<' not supported between instances of 'complex' and 'int'
>>> min(12, 12+9j)
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    min(12, 12+9j)
TypeError: '<' not supported between instances of 'complex' and 'int'
```

PROGRAMMING WITH PYTHON

operator

- Python has predefined functions for many mathematical, logical, relational, bitwise operations under the module “operator”.

```
>>> import operator as o
```

PROGRAMMING WITH PYTHON

operator

-
```
>>> o.add(45,34)
79
>>> o.sub(23,12)
11
>>> o.mul(23,45)
1035
```

```
>>> o.truediv(30,3)
10.0
>>> o.floordiv(35,2)
17
>>> o.mod(23,4)
3
>>> o.pow(3,4)
81
```

PROGRAMMING WITH PYTHON

Recap

- lambda()
- map()
- reduce()
- Filter()
- Max()
- Min()
- Zip()
- operator

THANK YOU

Lekha A
Computer Applications